7. Aufgabenblatt zur Vorlesung

Compilerbau WiSe 2023/2024

PD Dr. Fabian Stehn **Abgabe**: 26. Januar 2024

Aufgabe 7.0: Handwerk

Betrachten Sie die folgende, bereits erweiterte Grammatik G_1:

$$
\begin{aligned}
G_1 = (&N_1, \Sigma_1, P_1, S'),\\
N_1 &:= \{S', S, A, B, C\},\\
\Sigma_1 &:= \{\mathbf{a}, \mathbf{b}, \mathbf{c}\},\\
P_1 &:= \{\ S' \to S \quad (\#0)\\
&\qquad S \to \mathbf{a}\ A\ C \quad (\#1)\\
&\qquad\quad \mid C \quad (\#2)\\
&\qquad A \to C\ B \quad (\#3)\\
&\qquad\quad \mid \mathbf{a} \quad (\#4)\\
&\qquad B \to \mathbf{c}\ C \quad (\#5)\\
&\qquad\quad \mid \mathbf{b} \quad (\#6)\\
&\qquad C \to \mathbf{a} \quad (\#7)\ \}
\end{aligned}
$$

(a) Erstellen Sie den $\textsc{Follow}_1$-Mengen Graphen für G_1.

(b) Geben Sie alle $\textsc{Follow}_1$-Mengen der Nichtterminale aus Grammatik G_1 an.

(c) Ist G_1 **LL**(1)-konform?

(d) Erstellen Sie den **CLR**(1)-GoTo Automaten für Grammatik G_1

(e) Überführen Sie Ihre Lösung in eine LR-Parsetabelle

Hinweis:

- Sollten Konflikte vorliegen, so tragen sie alle Aktionen in die entsprechende Zelle der Tabelle ein!

(f) Ist G_1 **CLR**(1)-konform?

(g) Ist G_1 **SLR**-konform?

Aufgabe 7.1: Dies und Das

(a) Betrachten Sie folgenden Ausschnitt einer attributierten Grammatik:

Produktion	semantische Regeln
$A \to B$	$A.i := 11; B.x := 2 * A.v;$
$A \to B\ C$	$A.i := 22; B.x := 3 * A.v; C.y := A.y + B.y;$

Faktorisieren Sie obige Produktionen und passen Sie die semantischen Regeln entsprechend an.

(b) Betrachten Sie folgende Grammatik $G_+ = (\{E, T\}, \{+, \mathbf{int}, \mathbf{real}\}, P_+, E)$ mit den Produktionen

$$
\begin{aligned}
P_+ := \{\ E &\to E + T\\
&\mid T\\
T &\to \mathbf{int}\\
&\mid \mathbf{real}\ \}
\end{aligned}
$$

Die Worte der durch G_+ definierten Grammatik enthält Summen deren Summanden entweder ganze Zahlen (**int**) oder reelle Zahlen (**real**) sind. Erweitern Sie die Produktionen um semantische Regeln um das Attribut $E.type$ zu setzen: Ist kein Summand eine reelle Zahl, so ist das Attribut auf „integer“ zu setzten, anderenfalls auf „real“. Sie dürfen weitere Attribute einführen.

Aufgabe 7.2: Attributierte Geometrie

Hinweis: Dies ist eine (von drei) Aufgaben der Klausur im WS2017/2018

Ein Polygon ist eine ebene geometrische Figur, die durch einen geschlossenen Streckenzug begrenzt ist, siehe Abbildung 1. Die Endpunkte der Strecken des Streckenzuges werden Ecken genannt.

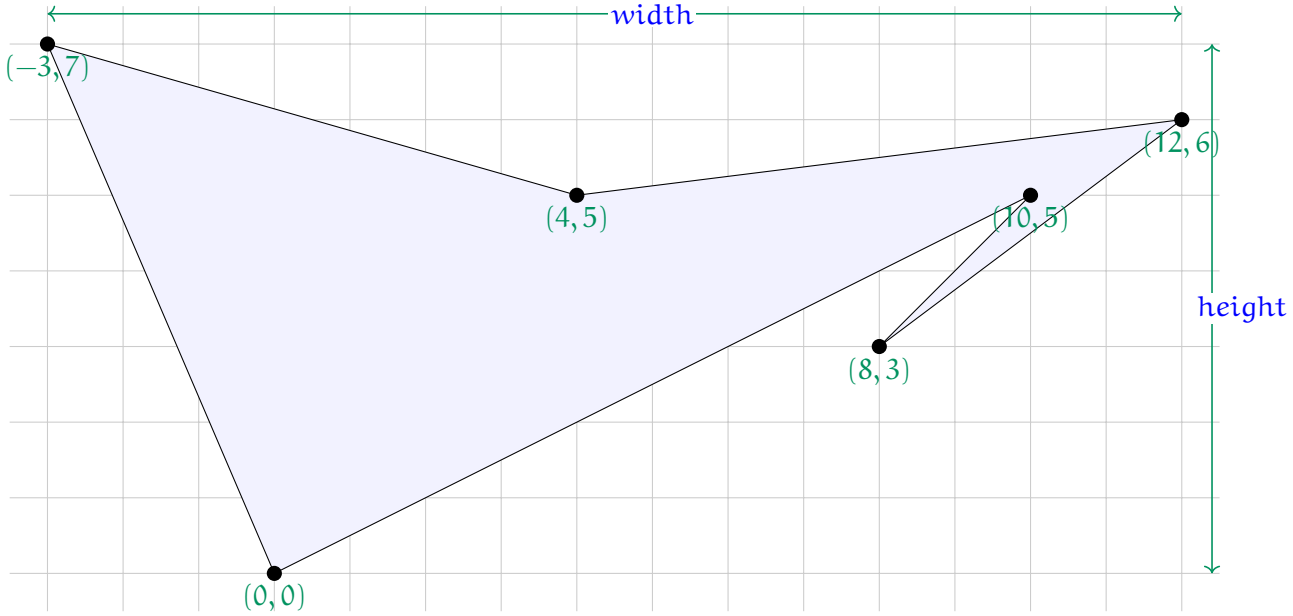


Abbildung 1: Ein Polygon mit 6 Ecken, einer Gesamtbreite von 15 und einer Gesamthöhe von 7.

Mit folgender Grammatik G können Polygone beschrieben werden:

$G = (N, \Sigma, P, P)$,

$N := \{P, L, C\}$,

$\Sigma := \{\textbf{polygon}, \textbf{num}, (,), ,\}$,

$P :=$ {

$P \rightarrow \textbf{polygon}\ L$ (#1)

$L \rightarrow C\ L_1$ (#2)

$\mid\ C$ (#3)

$C \rightarrow (\textbf{num}_1, \textbf{num}_2)$ (#4) }

Das Nichtterminal C codiert eine Ecke des Polygons als Tupel, wobei der erste Eintrag die x- und der zweite die y-Koordinate der Ecke repräsentiert. Das in Abbildung 1 gezeigte Polygon lässt sich beispielsweise durch die Zeichenfolge

polygon (0,0) (10,5) (8,3) (12,6) (4,5) (-3,7)

darstellen.

(a) Überführen Sie G in eine attributierte Grammatik, welche folgende Attribute setzt:

P.numV: die Anzahl Ecken des Polygons,

P.perimeter: die Gesamtlänge des das Polygon definierenden Streckenzuges (also die „Länge des Randes“ des Polygons).

P.width: die Gesamtbreite des Polygons.

P.height: die Gesamthöhe des Polygons.

Das Symbol **num** hat ein Attribut **num**.lexVal, welches ein `float`-Wert enthält und in der lexikalischen Analyse gesetzt wurde.

Geben Sie an, für welche Symbole Sie in Ihrer Lösung darüberhinaus Attribute verwenden: Benennen Sie die Attribute und beschreiben Sie deren Funktion.

Hinweis: Semantische Aktionen dürfen eine Mischung aus Pseudo-Code und arithmetischen Ausdrücken sein, solange diese eindeutig und allgemein verständlich ist. Verwenden Sie **keine** globalen Variablen um Werte zu speichern (alle Daten sollen in den Attributen gehalten werden).

(b) Ist Ihre Lösung S-attributiert?

(c) Ist Ihre Lösung L-attributiert?

(d) Zeichnen Sie den Syntaxbaum T_w für die Tokenfolge w, die einer Eingabe

polygon (0,0) (10.5,0) (10.5,10.5) (0,10.5)

entspricht.

Hinweis: Aufgaben **(b)-(d)** dürfen in einem Diagramm beantwortet werden, sofern dieses gut lesbar ist.

(e) Erweitern Sie T_w zu dem Abhängigkeitsgraphen A_{T_w} der Attribute Ihrer Lösung.

(f) Annotieren Sie A_{T_w} mit den konkreten Werten der gesetzten Attribute und werten Sie alle Attribute aus.

Hinweis: Passen Sie Ihre Attribute und semantischen Regeln entsprechend an.

Aufgabe 7.3: Ableiten ohne Leiden

Gegeben ist die Grammatik $G_{f'}$, welche eine Teilmenge aller Funktionen mit höchstens einer ungebundenen Variablen (x) beschreibt:

$G_{f'} = (N, \Sigma, P, E)$,

$N := \{E, T, F\}$,

$\Sigma := \{\mathbf{x}, \mathbf{num}, +, *, (,)\}$,

$P :=$ { E → E+T (#1) | T (#2)

T → T*F (#3) | F (#4)

F → (E) (#5) | **num** (#6) | **x** (#7) }

Hinweis: Das Token **x** beschreibt die ungebundene Variable, das Token **num** eine Konstante.

(a) Erweitern Sie $G_{f'}$ zu einer attributierten Grammatik, sodass zu Ausdrücken aus $L(G_{f'})$ die erste Ableitung nach **x** berechnet wird. Bei der Ableitung muss keine Vereinfachung vorgenommen werden.

Jedes Nichtterminal $X \in \{E, T, F\}$ sei mit folgenden Attributen versehen:

X.diff die Zeichenfolge (String), welche die erste Ableitung der durch X definierten (Teil-)Funktion darstellt,

X.exp die Zeichenfolge (String), welche die durch X definierte Funktion selbst darstellt.

Beide Attributwerte seinen vom Typ „String“, Konkatenation von Strings kann durch einfaches Hintereinanderschreiben ausgedrückt werden. Die finale Ableitung des gesamten Ausdrucks (also des geparsten Wortes), soll nach der Analyse in dem Attribut diff der Wurzel des Syntaxbaumes abgelesen werden können.

Welche Attribute müssen für welche Token zusätzlich in der lexikalischen Analyse gesetzt werden (benennen Sie die Attribute sinnvoll)?

Beispiel 0.1

Für das Wort $x \in L(G_{f'})$ hat nach der Auswertung die Wurzel (E) des Syntaxbaumes dieses Wortes die Attribute E.expr=„x“ und E.diff=„1“.

(b) Geben Sie den Datenflussgraphen für den Ausdruck 2*(x+1) an.
Hinweis: Geben Sie zunächst die Tokenfolge für dieses Wort, sowie gegebenenfalls die Attribute dieser Token an (sofern sie in der lexikalischen Analyse gesetzt wurden).

(c) Implementieren Sie Ihre Lösung mittels `Lex` und `Yacc`. Ihre Lösung soll Funktionen Zeilenweise einlesen pro Zeile die erste Ableitung der entsprechenden Funktion ausgeben (oder eine Fehlermeldung, falls eine Zeile keine wohlgeformte Formel enthält), siehe folgendes Beispiel:

```
root@localhost:~/yacc# ./deriver
2*x+1
  f(x)=(2)*(x)+1
  f'(x)=(2)*(1)+(x)*(0)+0
x
  f(x)=x
  f'(x)=1
x*x*x+2*x
  f(x)=((x)*(x))*(x)+(2)*(x)
  f'(x)=((x)*(x))*(1)+(x)*((x)*(1)+(x)*(1))+(2)*(1)+(x)*(0)
exit
  bye!
root@localhost:~/yacc#
```

Sie können annehmen, dass die Stringrepräsentation einer Funktion aus nicht mehr als 50 Zeichen besteht.

Hinweis: Fügen Sie Ihrer Lösung eine Datei `howToCompile.txt` bei, welche genau beschreibt, wie Ihre Lösung zu kompilieren ist.